

ICS-Sniper: A Targeted Blackhole Attack on Encrypted ICS Traffic

Gargi Mitra*, Chanyuan Liu*, Pritam Dash*, Elaine Yao[†], Alain Zhiyanov[‡], Aastha Mehta[‡] and Karthik Pattabiraman*

*Department of Electrical and Computer Engineering, University of British Columbia, Vancouver, Canada

[†]Department of Computing and Information Science, Cornell University, New York, USA

[‡]Department of Computer Science, University of British Columbia, Vancouver, Canada

Email: {gargi@ece, chanyuanl@ece, pdash@ece}.ubc.ca, elainey@cs.cornell.edu, {azhiyano@student, aasthakm@cs}.ubc.ca, karthikp@ece.ubc.ca

Abstract—Modern industrial control systems (ICS) increasingly host their Supervisory Control and Data Acquisition (SCADA) services in the cloud to reduce the costs of large-scale automation. To protect site-SCADA communications, ICS operators commonly use VPN tunneling and standard security practices. We show that, despite these security measures, an on-path Internet adversary can disrupt ICS operations without infiltrating the ICS perimeter, breaking encryption, or knowing the control logic. We present ICS-Sniper, a targeted blackhole attack that analyzes the VPN traffic metadata (sizes, direction, timing of packets) to identify narrow time windows, called *critical superperiods*, during which the site-SCADA traffic would likely contain highly critical commands or data. Post analysis, in a subsequent operational cycle, ICS-Sniper drops a small set of payload-carrying packets in the critical superperiods to disrupt the ICS's operations. We demonstrate three attacks on two realistic modern Secure Water Treatment (SWaT) plant testbeds that can potentially violate the operational safety of the ICS while evading state-of-the-art ICS attack detectors.

I. INTRODUCTION

Industrial control systems (ICS) automate critical processes such as power generation, water treatment, and manufacturing [1]. Attacks on these systems cause service outages, equipment damage, or safety hazards, resulting in productivity losses in downstream processes, civilian inconvenience, and reputational and financial losses for the parent organization [2], [3]. At the core of an ICS is the operational technology (OT) network, comprising Programmable Logic Controllers (PLCs), sensors, and actuators. In large plants, the physical process is divided into sub-processes that operate in tight synchronization to ensure safe and reliable operation. Each sub-process comprises multiple PLCs, one of which is designated the

[†]Elaine Yao was a Master's student at the Department of Electrical and Computer Engineering, UBC, when she participated in the research presented in this paper.

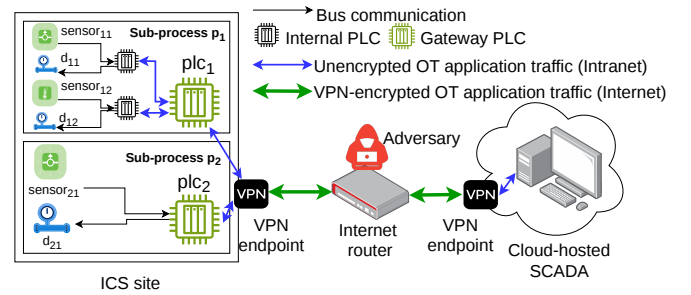


Fig. 1: A simple example of a modern OT network with cloud-hosted SCADA, and the position of ICS-Sniper.

gateway PLC and aggregates that sub-process's data. A central Supervisory Control and Data Acquisition (SCADA) software facilitates cross-sub-process synchronization by collecting data from gateway PLCs, serving cross-sub-process data requests, and issuing control commands. Timely delivery of these messages is critical for the physical process.

Historically, OT networks were confined to private intranets, minimizing exposure to remote adversaries. However, as industrial plants scaled and operations became geographically distributed, on-premise SCADA systems and Intranet connectivity across the plant grew increasingly inconvenient and expensive [4]. This has driven a gradual shift toward hosting SCADA systems on cloud platforms, offering cost savings, remote monitoring capabilities to operators, and built-in resilience for power and network connections [4], [5]. Cloud technologies are therefore expected to become critical to OT infrastructure in the coming years.

Migrating the SCADA to the cloud necessitates site-to-SCADA communications to traverse over the public Internet. To protect these communications from remote adversaries, including Man-In-The-Middle (MITM) attackers, the US NIST and AWS make four recommendations for ensuring the security of Internet-connected OT networks [6], [7]: (i) access control and message integrity through endpoint authentication and cryptographic mechanisms, (ii) implementing secure access

control with firewalls and intrusion detection systems (IDS), (iii) ensuring communication confidentiality through Virtual Private Networks (VPNs) tunneling, and (iv) maintaining high availability by employing early detection of volumetric Denial of Service (DoS) attacks and using redundant OT components.

A novel targeted DoS attack. We show that despite these security measures, the migration of the SCADA to the cloud makes ICSes susceptible to a new class of Denial of Service (DoS) attacks that exploit the expanded attack surface of modern ICSes. We introduce ICS-Sniper, a novel *targeted* blackhole DoS attack that breaks the synchronization among sub-processes by selectively dropping a *small number of critical OT messages* communicated between the cloud-hosted SCADA and on-premise PLCs, thereby causing significant disruption and damage to the ICS. To do so, ICS-Sniper neither infiltrates the ICS network nor breaks traffic encryption to access the payload. Instead, *it relies solely on the metadata of VPN-encrypted plant-to-SCADA traffic.*

ICS-Sniper operates from an on-path network vantage point outside the plant perimeter, and does not know about the plant’s control logic (see Figure 1). Yet, it can identify critical packet sequences within a narrow time window, and can cause disruptions by dropping 20% or 100% of packets in those windows, depending on the communication protocol used. This enables the adversary (e.g., rogue insiders in an Internet Service Provider (ISP), and enemy nation-state actors with ISP-level access) to perform the attack stealthily, evading conventional DoS attack detectors.

ICS-Sniper employs a novel encrypted traffic analysis (ETA) technique [8] to identify the critical packet sequences solely from traffic metadata, such as the sizes, timing, and direction of transmitted packets. For this purpose, it exploits two structural properties of ICS communication. First, each sub-process follows a state-transition model, where a state represents a unique combination of the mechanical device configurations. A sub-process transitions between states in response to messages from remote sub-processes. Second, each PLC in a given state exchanges a characteristic, periodically repeating message sequence with the SCADA, producing periodic, bursty traffic, with burst patterns changing detectably when a state transition occurs. Therefore, dropping packets close to the state transition time of a PLC breaks synchronization among sub-processes, pushing the ICS into an unsafe state.

ICS-Sniper’s core task is therefore to predict when these state transitions (or burst pattern changes) are imminent. The key challenge in inferring this is that often multiple gateway PLCs share a common VPN gateway, producing interleaved traffic streams that obscure these per-PLC periodic burst patterns. ICS-Sniper addresses this by decomposing the observed VPN traffic into “superperiods”—repeating composite patterns that capture fixed multiples of each constituent PLC’s periodic traffic patterns, from which it predicts upcoming state transition timings. ICS-Sniper identifies the superperiod by using phase folding [9], a technique from astronomical signal processing, which is particularly efficient in signal period determination in the presence of noise and frequently

changing burst patterns. For adapting this technique to ICS traffic, ICS-Sniper incorporates protocol-level knowledge of ICS standards (e.g., Modbus, ENIP), such as packet ordering, packet size ranges, and message transmission sequence, into its ETA technique. During the attack, ICS-Sniper drops payload-carrying packets in the superperiod immediately preceding a predicted transition and during the transition itself, maximally disrupting the physical process at the most opportune time.

To evaluate ICS-Sniper, we built two complementary validated testbeds based on the six-stage Secure Water Treatment (SWaT) plant [10]. The first testbed, which we refer to as “S-SWaT”, contains software-simulated PLCs, enabling fast prototyping. The second testbed, referred to as “H-SWaT”, uses real industry-standard PLCs for reproducing the timing and communication characteristics of real deployments. For both testbeds, we hosted the SCADA on a cloud VM, which communicates with PLCs over an OpenVPN tunnel [11]. The testbeds support industry-standard communication protocols, configurable communication periods and network conditions, network delay-tolerance mechanisms, and an ensemble [12] of DoS attack detectors. To our knowledge, these are the first testbeds providing a cloud-hosted SCADA over VPN.

Key results. 1) ICS-Sniper could infer the superperiod in the plant with an accuracy of 100% in both testbeds.

2) ICS-Sniper dropped critical packets at predicted superperiods for 14 minutes in S-SWaT. One attack delayed the start of the distribution of purified water by 15 minutes, while another caused a loss of purified water for 9 minutes 11 seconds. Similarly, a one-minute packet drop in H-SWaT caused a two-minute delay in the start of purified water distribution. Such disruptions would adversely affect downstream plants relying on the supplied water or inconvenience hundreds of users.

3) No ICS-Sniper attack drew alerts from both detector types before it impacted the ICS performance. These results collectively show that ICS-Sniper can achieve high-impact, low-visibility attacks even in secure ICS deployments.

Contributions. *To the best of our knowledge, we are the first to uncover an emerging threat that could affect cloud-connected OT networks and cause significant damage to an ICS.* Concretely, we make four contributions in this work.

1) Present ICS-Sniper, a *novel and practical ETA-based targeted DoS attack* that requires no knowledge about system internals, cryptographic secrets, or traffic payloads (§IV).

2) Provide two *realistic testbeds* simulating a modern water treatment plant for evaluating our attack as well as for conducting future security experiments (§V)¹.

3) Present a *comprehensive evaluation* of ICS-Sniper to highlight its efficacy in disrupting plant operations, and generalizability across various PLC configurations, communication protocols, and network conditions (§VI).

4) Highlight issues with existing DoS attack detectors, discuss countermeasures and challenges in adapting them to secure OT networks against ICS-Sniper (§VII).

¹The artifacts for this paper are available at <https://ics-sniper.github.io/ics-sec-website/>. These include detailed instructions for reproducing S-SWaT and H-SWaT, network traces captured from our testbeds, and ICS-Sniper’s code.

II. BACKGROUND

A. Cloud-hosted SCADA

The Supervisory Control and Data Acquisition (SCADA) system helps in centralized monitoring and control of geographically dispersed industrial assets [6]. PLCs at a site report process telemetry upstream to the SCADA, enabling plant operators to remotely monitor operations and detect anomalies. The SCADA transmits coordination data downstream to synchronize activities across sub-processes.

The migration of SCADA from the plant site to the cloud is driven by cost and operational benefits. This is especially valuable for industries managing large-scale infrastructure at remote sites with limited staff. Cloud providers further reduce operational downtime through high-availability guarantees. Industrial surveys confirm a recent surge in cloud-hosted SCADA adoption [13], with water and wastewater utilities across the USA, Canada, Mexico, Australia, and New Zealand among early adopters [14], [5], [15].

PLC-SCADA Communications. Large-scale ICSes employ a hierarchical communication architecture to minimize Internet-bound data flow and limit adversarial exposure to sensitive operational data. Direct communication with the cloud-hosted SCADA is not permitted for every device. Instead, if a sub-process has multiple PLCs, it designates a single gateway, either a PLC or a Remote Terminal Unit (RTU), as its sole aggregation point for upstream communication² (see Figure 1). The remaining PLCs within that sub-process execute local control logic and communicate exclusively with peer devices and the gateway device over the internal network. The typical number of gateway PLCs within the premises of large plants is ≈ 10 – 20 , and they are typically split across multiple VPN tunnels. For ICS-Sniper, disrupting one shared tunnel suffices. *We focus only on the communication between the gateway PLCs and the SCADA.*

B. Security of Modern OT Communications

OT communications over the Internet must contend with two classes of issues: (i) delays and drops due to network congestion or variable link speeds, and (ii) security threats such as MITM and DoS attacks. ICS designers address the former through two delay-tolerance mechanisms. First, legacy protocols such as Modbus [16] and CIP/ENIP [17] interface with the TCP/IP stack, relying on TCP retransmission to recover lost or delayed packets [18], [19], [20]. These protocols predominantly use synchronous communication, i.e., each PLC issues a new request only after receiving a response or timing out [21]. Second, if messages are lost or delayed, sub-processes continue local computations using the last received data until retransmission succeeds [22].

Legacy OT protocols have been upgraded to secure variants such as Secure Modbus [23] and CIP/ENIP-over-TLS [24] to protect ICS traffic. VPNs provide an additional layer of confidentiality by obfuscating endpoint addresses as well. ICSes commonly use them to secure site-to-SCADA communication

by routing OT traffic between statically addressed VPN gateways [25]. A site-to-SCADA VPN tunnel can be shared by multiple gateway PLCs. These OT VPN tunnels are not shared with the IT network due to security and performance concerns (as per NIST SP 800-82 recommendations) [6]. ICSes are further equipped with anomaly detectors based on process logs [26], [27], [28] and network traffic [29], [30].

C. Phase Folding Technique

Phase folding is a technique for uncovering hidden periodic patterns in noisy time series data of events. In the context of time series analysis and phase folding, an event is a discrete, instantaneous occurrence of a feature recorded at a specific point in time. Given a time series of events, the technique works as follows. First, it computes the time difference between every possible pair of events, and then tests each difference as a candidate period. For each candidate period T , this operation “folds” the timeline onto a single interval of length T , causing events separated by integer multiples of T to overlap along the folds. If T matches the true underlying period, 3 or more recurring events would align consistently after folding. In contrast, incorrect candidate periods would produce dispersed or inconsistent event alignments. The number of aligning events produces the alignment score. The candidate yielding the highest alignment score is identified as the true period. In the event of a tie, the one spanning the maximum length of the time series is selected as the true period.

III. PRELIMINARIES

We start with an example of an ICS and then describe how we represent an ICS as a state-transition system, which is crucial for understanding the design of ICS-Sniper (§IV-A).

ICS example. Consider the 2-stage water treatment plant in Figure 2, with two sub-processes: p_1 for raw water collection and p_2 for chemical treatment, synchronized through a cloud-hosted SCADA. Sub-process p_1 contains a pump pmp_1 , a water inlet valve vlv_1 , a water level sensor lvl_1 , and a gateway PLC plc_1 . Sub-process p_2 also contains a pump pmp_2 , a water inlet valve vlv_2 , a water level sensor lvl_2 , a flowmeter flw_2 , and a gateway PLC plc_2 . The pumps can be either On or Off, the valves can be Open or Close, and the water level sensors output a Low or High value, based on whether the water level is below or above a minimum and maximum threshold, respectively; flw_2 outputs the volume of treated water pumped out of pmp_2 since the start of the day. Each sub-process also contains a tank ($Tank_1$ and $Tank_2$) for holding water, and a pipeline connects them both. The plant works for 10 hours a day and must treat W gallons of water within this time, requiring timely coordination between plc_1 and plc_2 via the SCADA. The communication periods of plc_1 and plc_2 are 50ms and 40ms, respectively.

Figure 6 in Appendix A shows the control logic for p_1 ($P1_Control$) and p_2 ($P2_Control$). At the start of the operational cycle, all pumps are off, valves are closed, water level sensors are Low, and the flowmeter is set to 0. At a scheduled daily time, the PLCs are automatically turned on,

²RTUs and PLCs use similar communication protocols.

and the SCADA starts the operations by setting the global *plantOn* signal to 1 and sending it to the PLCs, which then open their water inlet valves. Once the water level in *Tank1* exceeds the minimum level, it turns on *pmp₁* and starts filling *Tank2*. Once *Tank2* is full (*lv₂*=High), *plc₂* closes *vlv₂*. At that point, *plc₁* must also turn *pmp₁* off within a bounded delay to prevent damage to the pipeline due to additional pressure. Thus, device configurations in *p₁* are influenced by remote events in *p₂*. Both PLCs read the value of *plantOn* from the SCADA; PLC *plc₁* also reads the value of *vlv₂* from the SCADA over the Internet. The PLCs read the rest of the local sensor values over their local network. To enable this, *plc₁* periodically sends data requests to SCADA.

A. ICS as a State Transition System

Local and global state transition systems. Over a complete operational cycle, each sub-process moves through a sequence of states, each state being a distinct device configuration with its corresponding control logic. A transition occurs whenever the control logic modifies one or more configurations. Because the ICS comprises multiple interacting sub-processes, it also evolves through a series of global states, each a unique combination of sub-process states. Figure 2 shows the state transition systems of our two example sub-processes, with states and transition conditions matching the physical and control conditions described in the example. Note that *p₁*'s transition from State 1 to State 2 is triggered by a remote event, making it a transition of interest to ICS-Sniper. Crucially, the correctness of global state transitions depends not only on individual sub-processes, but also on the timely exchange of time-critical information between sub-processes.

Safe and unsafe global states. Safe operation requires all sub-processes to stay synchronized, and only some combinations of sub-process states are safe, as dictated by the system's design, control logic, and timing constraints. In our example, the combinations $\{p_1 \text{ state} = 1, p_2 \text{ state} = 1\}$ and $\{p_1 \text{ state} = 2, p_2 \text{ state} = 2\}$ form the only two safe global states, as indicated in Figure 2. Any other combination, for instance, $\{p_1 \text{ state} = 1, p_2 \text{ state} = 2\}$ would be unsafe for the plant, because if the valve in *p₂* closes but the pump in *p₁* fails to stop within the required time window, excessive pressure may build up in the pipeline, causing it to burst. Changes in *p₂* must therefore be relayed to *p₁* within a bounded time window.

Periodicity of communications. Each PLC runs two concurrent cycles: a scan cycle that reads inputs, executes control logic, and updates outputs; and a communication cycle that handles all I/O, including *site-to-SCADA communications*. The scan-cycle period is set automatically from the minimum control-logic execution time, whereas the communication-cycle period is configured explicitly by programmers. Since PLC communications directly affect cross-PLC synchronization and demand deterministic timing, they are assigned a fixed period throughout the operational cycle. This is essential for synchronized and predictable system behavior [31]. To tolerate network variability, this period is set slightly above the worst-case communication delay observed during the development

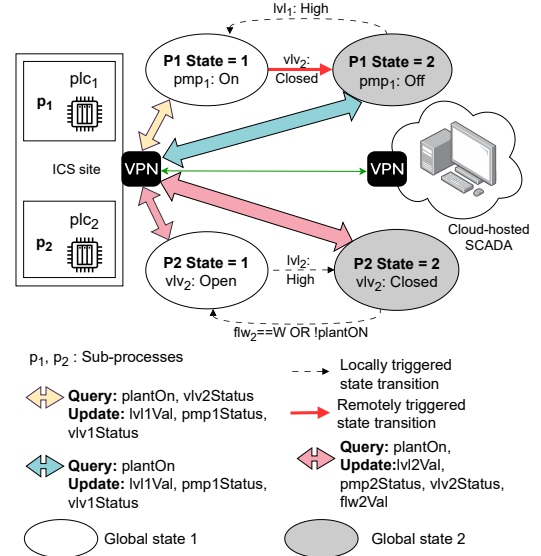


Fig. 2: Operations and communication of the sample ICS (§III) as a state-transition system.

and testing stage. For instance, if a PLC sends x messages per cycle, each with an average RTT of y ms, its period is configured as $(xy + \delta_t)$ ms, where δ_t is a jitter margin derived from observed network jitter. If a communication cycle completes early, the PLC remains idle until the period expires before initiating the next cycle. A PLC's communication period determines its tolerance for communication delays, and hence, the delay tolerance of its state transitions. A delay in scheduled state transition might break inter-sub-process coordination, jeopardizing ICS performance and safety [6].

The volume and content of messages sent by a PLC to the SCADA may vary across process states. Consequently, each sub-process state, and therefore, each global state, corresponds to a distinct set of network communications. In the example shown in Figure 2, p_2 exchanges the same set of messages across both its states, but there is one extra message in the first state of p_1 as compared to the second.

B. Characteristics of OT VPN traffic

To characterize PLC traffic over a VPN tunnel, we implemented the example control logic on a real PLC and our cloud-hosted SCADA, and observed the network traffic between them. Three key observations guide the design of ICS-Sniper.

O1. Number and ordering of packets: PLC-SCADA communications use either a connected or unconnected protocol variant. In the connected variant, a persistent TCP connection carries each update as a single PLC request and a single SCADA response. In the unconnected variant, each update opens a new TCP connection and comprises a fixed sequence of 10 packets sent in strict order. Under heavy bursts, the gateway's TCP layer may coalesce encrypted packets into one TCP segment, though an adversary can still infer their number and sizes from VPN-layer headers [11]. Across both variants,

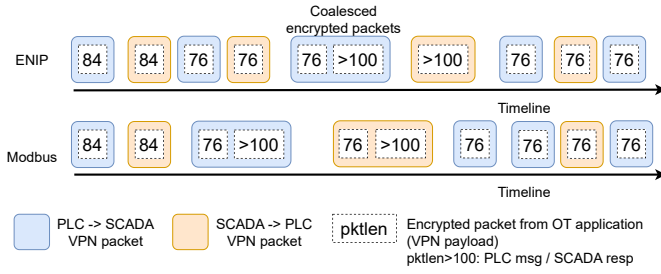


Fig. 3: Traffic patterns of unconnected protocol variants.

the request sequence within a state may vary across cycles, but communication is serialized. After each request, the PLC waits for the SCADA's response before sending the next.

O2. Packet size sequence: The sequence of sizes of the 10 encrypted packets in the unconnected variants, mentioned in **O1**, exhibits protocol-specific patterns, as shown in Figure 3. The size of the first packet is always 84 bytes. Of the 10 packets exchanged, 6 are sent by the PLC to the SCADA and 4 in the reverse direction. Only two packets carry the application payload³. For ENIP, these packets are larger than 100 bytes, and for Modbus, they are larger than 85 bytes. The payload packets in the connected variants have similar sizes.

O3. Packet size variability: Modbus allows aggregating multiple PLC requests into one packet to cut round-trip overhead and latency, which makes packet sizes vary with request grouping. ENIP does not support aggregation but uses symbolic (tag-name) addressing. Since tag names are variable-length strings, packet size depends on the referenced tag name, introducing size variability even without request aggregation.

C. Threat Model and Assumptions

Adversary position and capabilities. We consider an adversary with covert, persistent control of one or more ISP or provider-edge routers on the path between the plant gateway and the cloud-hosted SCADA. From there, the adversary can (i) observe traffic metadata such as unencrypted VPN packet headers (IP/TCP), packet lengths, and timings; (ii) record flows for offline analysis; and (iii) drop or delay packets. However, the adversary cannot break encryption, read or inject payloads, or compromise any ICS-internal device.

Adversaries have compromised ISP or provider-edge routers in the past by exploiting zero-day or unpatched vulnerabilities, misconfigurations, or leaked credentials [32]. For instance, recently disclosed vulnerabilities in Cisco and Juniper routers [33], [34], [35] grant root access and code execution privileges to remote adversaries, enabling them to monitor traffic using tools such as Wireshark, as well as drop packets. This adversary might be a rogue ISP or an enemy nation-state actor who leverages background knowledge to compromise routers known to carry ICS traffic.

Once in position, the adversary isolates the VPN flow carrying ICS traffic. This is straightforward when the endpoint

³The remaining packets are used for connection establishment and tear-down.

addresses of the VPN tunnel are known: in OT networks, these addresses are long-lived, and a site-to-site tunnel is dedicated to plant-to-SCADA communication, isolated from IT and other-organization traffic for security and performance concerns (NIST SP 800-82) [6]. Otherwise, the adversary proceeds in two steps: separating VPN from non-VPN flows, then ICS from non-ICS VPN flows. The first step is straightforward if the VPN tunnel uses default ports (e.g., 1194/443 for OpenVPN). When it does not, the adversary applies known fingerprinting techniques [36], [37] that rely only on packet direction, size, and ordering, all visible at a compromised router. In OpenVPN, for example, the control messages exchanged during connection bootstrapping or session key renewal produce distinctive packet-size sequences [36]. Our testbeds described in §V-B use `tls-crypt-v2`, which leaves these fingerprints intact. For the second step, the adversary identifies the ICS flow among the isolated VPN flows by its balanced bidirectional traffic, stable timing, periodic bursts, and long-lived endpoints. Since OT tunnels carry no other organizational traffic, these features persist despite VPN tunneling.

Adversary's background knowledge. The adversary knows nothing of the target's system design, control algorithm, or implementation. It *knows only the general properties of ICSes*, e.g., that communications are repetitive and periodic, and that sub-processes perform a fixed set of actions.

D. Scope

ICS-Sniper targets batch-processing ICSes that exchange regular, periodic (synchronous) updates with the SCADA. These systems constitute a large class of safety-critical systems spanning water and wastewater treatment and discrete manufacturing [38], [39]. We focus on synchronous communication because, when the site updates the SCADA continuously and only a small fraction of packets are critical, it is difficult to separate the critical and non-critical packets. This is the core problem ICS-Sniper solves. Asynchronous communication, where the site updates the SCADA only on a state change, makes this trivial; so an adversary would not need ICS-Sniper.

IV. ICS-SNIPER

We start with a high-level overview of ICS-Sniper (§IV-A), followed by the details of its two phases of operation (§IV-B–IV-C). Finally, we include a discussion of the generalizability of ICS-Sniper (§IV-D). We discuss in Appendix C how ICS-Sniper is better than known volumetric DoS attacks.

A. High-Level Overview of ICS-Sniper

Premise. As discussed in §III-A, each PLC generates a periodic bursty packet flow throughout an operational cycle, with each burst carrying data requests for remote sub-processes and its own process updates to the SCADA. The number and sizes of packets within a burst characterize the PLC's current state, and a change in burst pattern signals a state transition. Without knowing the plant's control logic, ICS-Sniper operates on the intuition that *a change in a burst pattern indicates a state transition in the corresponding sub-process, triggered by one*

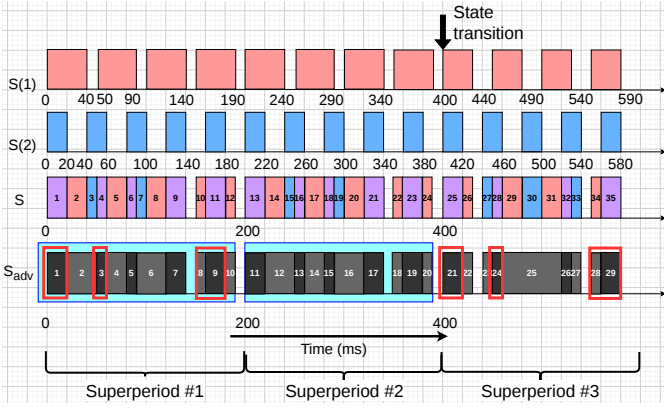


Fig. 4: Packet burst patterns for our example ICS. $S(1)$ and $S(2)$ represent bursts from plc_1 , and plc_2 , respectively. S denotes the interleaved burst pattern within the VPN tunnel, and S_{adv} is the adversary's view of the VPN flow.

or more packets in either the transitioning burst or the one immediately preceding it. ICS-Sniper's goal is to identify and drop these bursts to disrupt cross-sub-process synchronization.

Main challenge. In an ICS, multiple gateway PLCs might share a single VPN tunnel to the cloud-hosted SCADA, aggregating their packet streams into one encrypted flow. Because VPN encapsulation hides the per-PLC IP/TCP headers, it is challenging for the adversary to discern per-PLC traffic based solely on observable metadata of this aggregate flow.

Core Idea. To address this challenge, ICS-Sniper exploits a key insight: although VPN encryption obscures per-PLC traffic patterns, the periodic bursts of individual PLCs produce a structured aggregate flow characterized by alternating regions of overlapping (concurrent PLC activity) and non-overlapping (single or no active PLC) bursts. This structure gives rise to a periodic pattern in the aggregate flow, which we call *superperiodicity*. The sequence of overlap and non-overlap regions recurs with a fixed period, the *superperiod* L , equal to the least common multiple of the individual PLC communication periods. While ICS-Sniper cannot identify which specific sub-process undergoes a state transition, it can identify the superperiods during which one or more sub-processes do, and use this to isolate a small subset of packets critical to cross-sub-process synchronization, and selectively drop them.

Illustration. Figure 4 shows the traffic burst patterns of the two PLCs introduced in Figure 2: $S(1)$ for plc_1 (period 50 ms, initial burst length 40 ms) and $S(2)$ for plc_2 (period 40 ms, burst length 20 ms). We assume that plc_1 transitions from State-1 to State-2 after eight periods (i.e., after 400 ms), reducing its burst length from 40 ms to 30 ms as it transmits fewer packets in State-2. The burst duration of $S(2)$ remains unchanged since plc_2 exchanges the same set of messages with the SCADA in both states. S represents the aggregate traffic resulting from the multiplexing of the periodic bursts $S(1)$ and $S(2)$. The purple regions indicate time intervals where bursts from both PLCs overlap, causing their packets to appear

interleaved. The sequence of overlap and non-overlap regions recurs periodically, forming the superperiodic structure of S . Overlap regions 1-12 in S repeat in regions 13-24 with a period of 200 ms. Therefore, the superperiod of the aggregate traffic S is 200 ms ($= \text{LCM}(50 \text{ ms}, 40 \text{ ms})$). After 400 ms, although a change in a PLC's burst pattern may alter the appearance of some overlap regions, other regions continue to exhibit the same superperiodicity. In Figure 4, despite the change in $S(1)$ during its ninth period, regions 1, 4, 5, 8, 10, and 11 recur in regions 25, 28, 29, 31, 34, and 35 with a period of 200 ms. Therefore, even if state transitions alter the traffic patterns of some overlap regions within a superperiod, the remaining recurring overlap regions aid in determining the superperiod.

Similar to S , its encrypted version S_{adv} also consists of alternating overlap and non-overlap regions. However, in the absence of per-PLC identifiers, S_{adv} lacks certain structural periodic information that is otherwise visible in S . For example, regions 2 and 3 in S appear as one contiguous non-overlap region (region 2) in S_{adv} . Nevertheless, even under VPN encryption, portions of the traffic in S_{adv} still exhibit superperiodic recurrence. Overlap regions 1-10 in S_{adv} repeat in regions 11-20 with a periodicity of 200 ms (highlighted regions), exhibiting a superperiod of 200 ms ($= \text{LCM}(50 \text{ ms}, 40 \text{ ms})$). Even after the state transition of plc_1 , regions 1, 3, 8, and 9 in S reappear as regions 21, 24, 28, and 29, respectively, exhibiting a period of 200 ms.

ETA technique and associated challenges: ICS-Sniper leverages traffic metadata to identify recurring overlap regions and infer the underlying superperiodic structure and burst pattern changes. However, network noise and frequent state transitions can distort the overlap regions, making superperiod estimation challenging. ICS-Sniper addresses this challenge using phase folding (described in §II-C), a noise-resilient technique for detecting periodic structures. Specifically, it applies phase folding to three traffic metadata features (referred to as events): overlap region durations, inter-packet interval durations, and unique packet lengths. We refer to the three steps as $\phi_{overlap}$, $\phi_{interval}$ and ϕ_{pktlen} , respectively. First, in the $\phi_{overlap}$ step, ICS-Sniper applies phase folding to the coarse-grained feature overlap-region durations, which yields the superperiod value. However, noise in the overlap region durations can introduce errors in the estimated superperiod value (e.g., yielding a superperiod of 179.5s instead of 180s), which can accrue over time and significantly misalign the inferred boundaries of later superperiods in the operational cycle. Therefore, ICS-Sniper also applies phase folding to the finer-grained features, inter-packet timing intervals (IPTs) and unique packet lengths in $\phi_{interval}$ and ϕ_{pktlen} steps, respectively. These steps yield precise period values of one or more PLCs (e.g., 60s, 90s, etc.). ICS-Sniper determines the correct superperiod value as the least common multiple of the candidate periods generated in $\phi_{interval}$ and ϕ_{pktlen} that is closest to the superperiod estimated from $\phi_{overlap}$.

Stages: ICS-Sniper operates in two stages. In the *Process Profiling Stage*, it records OT traffic from a compromised

router and analyzes traffic metadata to infer the superperiod length L , and subsequently detects state transitions. In the *Active Attack Stage*, it monitors live traffic and drops critical packets at predicted times. If ICS-Sniper detects a change in burst pattern within the x^{th} superperiod, it drops packets in both the $(x-1)^{th}$ and x^{th} superperiods.

B. Process Profiling Stage

Step 1: Capture the network traffic for a complete operational cycle of the target sub-process. ICS-Sniper must record all VPN packets spanning a complete operational cycle, as missing packets can compromise accuracy. We discuss two challenges in this step and how ICS-Sniper addresses them.

C1: Identifying the first and last packets of a cycle. Batch-processing ICSes alternate between an active operational phase (high network activity) and an idle loading phase (negligible activity). In the example from §III, the plant operates for 10 hours followed by a 14-hour idle period. IPTs differ significantly between these phases, allowing ICS-Sniper to detect phase boundaries. If ICS-Sniper begins capturing at an arbitrary point, it would encounter one of the two scenarios. **(1) Mid-operational phase.** On observing an IPT that far exceeds the running average, ICS-Sniper infers the current cycle has ended, discards all prior packets, treats the next packet as a new cycle's start, and waits for the next large interval to mark its end. **(2) Idle phase.** ICS-Sniper detects an extended inactivity period followed by packets with low IPTs, and captures all subsequent packets until the next inactivity period signals the cycle's end.

C2: Handling packet variance due to natural network fluctuations and eliminating non-PLC packets. Using TCP sequence numbers, ICS-Sniper removes retransmitted VPN packets, and by parsing VPN headers (see **O1**) it extracts encrypted packet sizes, discarding any outside the protocol-specific ranges (see **O2**). It likewise discards heartbeat, keep-alive, and liveness-poll packets so they do not perturb the superperiod estimate; their single-byte payloads result in fixed sizes outside the protocol-specific ranges (**O2**), making them easy to identify. This is repeated over multiple cycles until a consistent packet count recurs for the majority, after which one representative sequence is retained for subsequent processing.

Step 2: Metadata extraction. For each packet in the captured trace, ICS-Sniper extracts the source and destination IP addresses, its arrival time relative to the beginning of the operational cycle, and the sequence of encrypted packet sizes (similar to Step 1), yielding a sequence S of tuples (l_t, t) . l_t is a signed integer representing an encrypted packet length (positive for PLC $\rightarrow$ SCADA and negative for SCADA $\rightarrow$ PLC) and t is the timestamp relative to the first packet in the operational cycle. Encrypted packets that were coalesced into the same VPN packet are assigned the same timestamp t . For determining which IP address belongs to the PLC and which one belongs to the SCADA, ICS-Sniper applies **O2** in §III-B for unconnected variants, i.e., IP addresses generating 60% of encrypted packets must belong to the site gateway, while IP addresses generating 40% of encrypted packets must

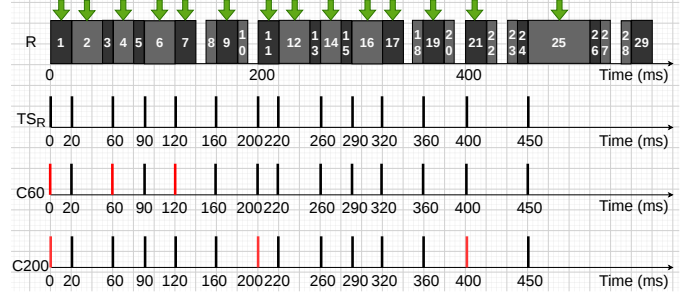


Fig. 5: Identifying stable regions and superperiods from overlap regions. The regions marked with green arrows indicate stable regions. TS_R is the resultant time series. The red lines in C_{60} and C_{200} show events that line up while phase folding with 60ms and 200ms candidates respectively.

belong to the SCADA. For connected variants, the IP address assignment is interchangeable.

Step 3: Identifying the superperiod L . To identify the superperiod L , ICS-Sniper processes the sequence S from Step 2 using phase folding [9] (described in §II-C). We discuss how ICS-Sniper applies this technique while considering the three traffic metadata features as events: overlap region durations, IPT durations, and unique packet lengths. For each event type, ICS-Sniper constructs two event series, based on traffic direction (PLC $\rightarrow$ SCADA and SCADA $\rightarrow$ PLC).

Step $\phi_{overlap}$: ICS-Sniper delineates overlap regions from shifts in IPTs. From S , it computes the IPT between each pair of consecutive packets, yielding a sequence S' . It then slides a 2-element window over S' , computing the absolute difference between each consecutive IPT pair. If this difference exceeds a threshold, a region boundary is placed between the two packets corresponding to the larger IPT difference. This boundary indicates a transition between an overlap and a non-overlap region, or between two overlap regions with differing numbers of concurrently active PLCs. However, this technique cannot distinguish between two non-overlap regions belonging to different PLCs, since all PLCs transmit messages at similar rates. The threshold is determined via hierarchical clustering on all consecutive pairwise IPT differences in S' , which partitions them into maximally separated groups. ICS-Sniper then selects the largest value in the smallest-difference cluster as the threshold, i.e., the maximum IPT variation for packets belonging to a single region. This yields a sequence of regions $R = \{(r_{start}, r_{end})\}$, where r_{start} and r_{end} are the start and end times of a region.

In our example shown in Figure 4, this step produces 29 regions for our ICS example continued from §IV-A. Of these regions, ICS-Sniper uses the longest ones for periodicity analysis, as they indicate stable regions. For each region in R , ICS-Sniper computes its duration as $(r_{end} - r_{start})$ and applies hierarchical clustering to the resulting set of durations to filter the long-duration regions that serve as reliable reference points for period estimation. Let us assume, in our example, all regions of duration ≥ 20 ms are considered stable. This

produces 14 stable regions, indicated by the green arrows in Figure 5. From their start times, ICS-Sniper builds a time series (TS_R in Figure 5), yielding 36 candidate periods. Since at least 3 events must align for a superperiodic structure, candidates exceeding one-third of the cycle length (200 ms here) are eliminated, leaving 18. Figure 5 shows phase folding for two candidates, 60 ms and 200 ms (red lines mark aligned events): 3 events align for both, but 200 ms spans more of the operational cycle. Evaluating the remaining candidates likewise identifies 200 ms as the superperiod in this example.

Step $\phi_{interval}$: Short PLC communication periods combined with network jitter can make stable region identification unreliable. ICS-Sniper therefore also phase-folds the IPTs directly, treating each as an event. It clusters the IPT values in S' by hierarchical clustering—maximizing separation while grouping similar intervals—so that each cluster captures the IPTs arising when a given number of PLC periods coincide, then builds a time series per cluster and phase-folds it as in the previous step. The period determined for each cluster would either coincide with the period of a PLC or the Least Common Multiple (LCM) of the periods of a subset of PLCs.

Step ϕ_{pktlen} : ICS-Sniper exploits the payload size variability of OT packets (**O3** in §III-B). If a PLC has at least one packet size recurring across multiple communication cycles, phase folding can recover its communication period. When this holds for all PLCs, the LCM of the recovered periods results in L . Otherwise, only a subset of periods is recovered. Because ICS-Sniper lacks ground truth, it always runs all three event types. Even partial length-based estimates help refine the coarse overlap-based estimate. ICS-Sniper identifies the unique packet sizes in S and, for each, builds a time series using the timestamps at which that size appears and phase-folds it. The recovered period matches the period of one or more PLCs transmitting packets of that length.

Step 4: Predicting state transitions. Once ICS-Sniper determines the superperiod L , it splits the metadata sequence S into sub-sequences of duration L and compares each adjacent pair of superperiods using two heuristics, each capturing a different aspect of traffic variation: (i) change in total payload packet count, which captures changes in the number of packets due to state transitions, and (ii) change in the number of unique packet sizes, indicating new message sequences due to state transitions. Individually, each heuristic is imperfect: network jitter and retransmissions can affect packet counts, while spurious control packets can introduce unexpected message sizes. However, a combination of heuristics serves as a strong indicator of a state transition. ICS-Sniper designates a superperiod as *critical* if either its total packet count or its number of unique message sizes, or both, differ from the corresponding values in the preceding superperiod. If multiple superperiods are designated critical, ICS-Sniper ranks them by first counting how many heuristics show a difference (one or both), and prioritizes the one with differences across the maximum number of heuristics. A tie is resolved by the combined magnitude of those differences, computed as the sum of the absolute differences in packet count and unique message sizes

relative to the preceding superperiod. The superperiod with the highest-ranked discrepancy is assigned the highest priority.

Corner case analysis. ICS-Sniper is likely to mispredict state transitions or fail to identify the superperiod under the following unusual yet theoretically feasible scenarios: transitions driven purely by local sub-process changes; concurrent transitions that leave aggregate traffic metadata unchanged; very frequent per-PLC transitions that obscure superperiod patterns; or a superperiod (the LCM of PLC periods) that is too large to repeat within the operational cycle. However, this is highly unlikely in practice, since real PLC periods range from a few milliseconds to seconds, while operational cycles span hours. Appendix B discusses these scenarios in detail.

C. Active Attack Stage

In this stage, ICS-Sniper drops the packets in the critical superperiods in real-time. It observes the traffic and waits to detect the start of a new operational cycle, identifying the first packet with the technique from Step 1 of the profiling stage (§IV-B). If the target is the x^{th} superperiod in S , ICS-Sniper drops payload-carrying packets in the $(x-1)^{th}$ and x^{th} superperiods, i.e., for an operational cycle starting at time T_s , within $[(T_s + (x-2) \times L), (T_s + xL)]$. Only application-payload packets are dropped: 20% of the packets in that window for unconnected protocol variants (**O2** in §III-B), and all packets in the connected variant, where every packet carries payload. Dropping payload packets across this window also drops all retransmissions, effectively preventing the state transitions. Appendix C compares this to volumetric DoS techniques.

D. Generalizability

Across protocols. ICS-Sniper depends only on the periodic request-response structure of ICS communication, not on any protocol-specific field. Any protocol that follows a request-response pattern with periodic updates, e.g., MMS in IEC 61850 [40], produces the same superperiodic, balanced traffic pattern that ICS-Sniper exploits. This pattern is also preserved when several similar protocols share a flow (e.g., Modbus + MMS, as in the EPIC power-grid dataset [41]).

Across VPNs. ICS-Sniper can be adapted to different VPNs with no changes. Every VPN protocol that secures site-to-SCADA traffic encrypts each inner IP packet 1:1, preserving the timing, ordering, and burst structure that ICS-Sniper relies on. Handshake and key-exchange packets, which do not carry process data, are identified and discarded by their characteristic fixed sizes so that they do not perturb the superperiod estimate. The only factor that might affect ICS-Sniper’s profiling accuracy is ciphers that coarsen packet size and thus reduce size resolution. In this case, ICS-Sniper falls back on timing alone, which we show is sufficient to recover the superperiodic structure (§VI-A).

V. EXPERIMENTAL SETUP

We first describe the original Secure Water Treatment (SWaT) plant setup at a high level [10]. We then describe our enhanced testbed that emulates a plant with SCADA deployed

on the cloud. We then describe the implementation of the attacks in our setup and also the attack detection techniques that we later use to evaluate ICS-Sniper’s efficacy.

A. SWaT Plant

We evaluated ICS-Sniper on testbeds modeled after the miniature 6-stage Secure Water Treatment (SWaT) plant at SUTD [10]. To our knowledge, this is the only openly available realistic multi-sub-process control logic, allowing us to evaluate ICS-Sniper’s impact on the plant’s operations. The plant purifies wastewater in six stages, each controlled by a sub-process: raw water storage ($P1$), chemical dosing ($P2$), ultrafiltration ($P3$), dechlorination ($P4$), reverse osmosis ($P5$), and backwash ($P6$). The testbed consists of six PLCs, each acting as the gateway of a sub-process. The number of internal PLCs behind each gateway does not affect our evaluation, as ICS-Sniper targets only gateway-PLC $\rightarrow$ SCADA traffic.

SWaT itself has six PLCs, all of which are gateways reporting to the SCADA. Where a real plant would place additional PLCs behind a gateway, SWaT represents that internal layer by a control logic, which acts on a simulated model of the mechanical components it governs. The resulting component actions drive the overall physical process, which is itself mathematically modeled. To maintain synchronization across all six sub-processes, the PLCs periodically exchange information about the current state of their physical components and other sub-processes with the SCADA system. The PLCs communicate with the SCADA over the Intranet using unencrypted communication channels. We provide additional description of the original SWaT plant in Appendix D.

B. Internet-connected SWaT Testbed Emulation

We built two testbeds that mimic a geo-distributed version of the SWaT plant, where the synchronization among its six sub-processes is maintained by a central cloud-hosted SCADA system that communicates with the PLCs over the Internet via an OpenVPN tunnel. Both testbeds allow configuring different PLC periods and simulating various network conditions.

(i) S-SWaT: Our first testbed contains software-simulated PLCs to enable fast prototyping. Unlike the original physical testbed [10], our emulation does not have any real mechanical devices. Instead, we use the control logic and physical process models available from an open-source single-host Python simulation of the same testbed [42]. We implemented the control logic of the 6 sub-processes and the physical process models as separate Python modules, and we execute them as separate processes within an Amazon EC2 VM, which represents the plant site. The traffic of all 6 PLCs is routed through the same network interface of the EC2 VM, resembling the plant site gateway. S-SWaT has an operational cycle of 4 hours.

For site-to-SCADA communication, S-SWaT provides configurable support for unconnected variants of two industry-relevant protocols: ENIP/TCP and Modbus/TCP. For securing the communications, we encapsulated the site-to-SCADA traffic inside an OpenVPN tunnel that uses TLS Crypt v2, AES-256-GCM for data encryption, and SHA-256 to verify message

integrity. This ensures adequate confidentiality, integrity, and authenticity of OT communications.

We deploy the testbed in the cloud using three Amazon EC2 VMs (see Figure 7 in Appendix D). VM-1 simulates the plant (ICS site) and runs the six simulated PLCs. VM-2 runs the SCADA system, which logs sensor readings every 10 seconds for telemetry and attack detection. VM-3 acts as an on-path compromised router, through which we route all OpenVPN packets between the site and the SCADA. This setup allows for analyzing the ICS traffic patterns in realistic network settings.

We consider an adversary who targets the site-SCADA communication. We place ICS-Sniper on VM-3 (Figure 7 in Appendix D) to emulate an adversary on the network path connecting the plant VM gateway to the SCADA VM gateway. ICS-Sniper relies on (i) a packet capturing tool (we use tshark [43]), (ii) a process profiling module implemented in Python by us, and (iii) a network controller module for dropping packets, implemented in Python by us.

(ii) H-SWaT: This testbed uses real industry-standard PLCs (Allen-Bradley Micro800 series) for reproducing the timing and communication characteristics of real deployments (validated by comparing with the traffic characteristics of the original SWaT testbed). We converted the Python-based control logic to structured text code and deployed it on the real PLCs using the Rockwell Automation Connected Components Workbench software. The 6 PLCs are connected to a gateway router that is connected to the Internet. For site-to-SCADA communication, H-SWaT uses connected Modbus/TCP encapsulated in an OpenVPN tunnel (see Figure 8 in Appendix D). H-SWaT is faster in processing and communication, and has an operational cycle of 2 hours, with the SCADA logging sensor readings every second. The rest of the testbed (physical process model, SCADA, simulated on-path compromised router, and attack execution infrastructure) is the same as S-SWaT. To the best of our knowledge, this is the first validated ICS testbed with SCADA in the cloud.

C. ICS Attack Detection Techniques

To our knowledge, there is no technique for effectively detecting targeted blackhole attacks on ICS OT traffic. Existing ICS attack detection techniques [30], [29], [26], [27], [28] were tailored for detecting volumetric DoS attacks from network traffic or anomalous sensor readings from process logs in real time. Nevertheless, we assess their capability to detect attacks performed by ICS-Sniper. Additionally, we use IPAL [12], an ensemble of these state-of-the-art ICS attack detectors. IPAL uses two relevant ensemble strategies: All and Any [12] to combine the alerts raised by the individual detectors mentioned above. They improve the False Positive Alarms (FPA) and False Negative Alarms (FNA) of the ensemble, respectively. As these ICS attack detectors are directly relevant to ICS-Sniper, we evaluate against them in §VI-C. However, conventional cloud-based attack detectors [44], [45], [46] are orthogonal to our threat model (see §VIII).

Choice of detectors. The detectors in the ensemble represent a diverse set of detection techniques and have been

evaluated in prior work on various ICS datasets, including SWaT [47] and WADI [48]. The network anomaly detectors work by analyzing the content and metadata of unencrypted network traffic captured at the SCADA, and the process anomaly detectors flag anomalies in the sensor and actuator values caused as a result of the ICS-Sniper attack. Each of these detectors considers different features to detect anomalies in ICS, thereby covering a broad range of features and techniques. Table I summarizes the detectors by their type, technique, and features used for anomaly detection.

To improve the performance of these detectors against ICS-Sniper, we adapted them to use only those features that are relevant to our setup. For instance, we configured *Kitsune* to detect anomalies in timing, source IP, and function code combinations instead of new communicating entity pairs.

VI. EVALUATION

We evaluate ICS-Sniper on three factors that influence its success, namely profiling accuracy (§VI-A), impact of attack (§VI-B), and stealthiness (§VI-C). We also assess its generalizability under multiple configurations of PLC periods, message transmission rates, scan cycles, and network conditions.

Table II lists the testbed configurations used in our evaluation. S-MD/EN-BASE and H-MD-BASE serve as the base configurations for S-SWaT and H-SWaT, respectively, using the smallest scan cycle and PLC communication periods that ensure correct process execution, as determined by industry best practices described in §III-A. S-EN-RNC repeats S-EN-BASE on a different day under natural network perturbations, and S-EN-PCP changes the last three PLC periods to 90 s. H-MD-LOSS1 is similar to H-MD-BASE with 1% random packet loss added to the network. H-MD-LOSS2 also simulates the same packet loss for a different set of PLC periods.

A. Profiling Accuracy

ICS-Sniper’s success in the active attack stage depends on its accuracy in the profiling stage. We assess profiling accuracy by measuring (i) accuracy in identifying superperiods, and (ii) effectiveness in identifying critical superperiods.

Experiment. For each configuration, we ran our SWaT testbed for one complete operational cycle, with ICS-Sniper capturing the encrypted VPN traffic at VM-3 (see Figure 7).

Estimating superperiods. Table V in Appendix E shows the actual and predicted superperiod for the seven configurations. We compute the accuracy of prediction as follows.

$$Accuracy = \left(1 - \frac{|SP_{pred} - SP_{true}|}{SP_{true}}\right) \times 100\%$$

where SP_{true} is the actual superperiod and SP_{pred} is the superperiod predicted by ICS-Sniper for a given setup.

Across all configurations, ICS-Sniper achieves an accuracy of 100% in identifying the superperiod, demonstrating generalizability. This holds even for H-MD-LOSS1 and H-MD-LOSS2 (different period configurations under aggressive packet loss), confirming ICS-Sniper’s robustness to lossy network conditions. This robustness stems from ICS-Sniper’s

use of two kinds of complementary metadata: packet size and inter-packet timing. Even when packet loss distorted the periodicities of the overlap regions ($\phi_{overlap}$) and IPT clusters ($\phi_{interval}$), each PLC still transmitted enough distinctly sized packets on schedule to preserve their periodicity, allowing ICS-Sniper to recover the superperiod from the ϕ_{pktlen} step. The hierarchical cluster boundaries are auto-derived from the level where the inter-cluster values have the largest separation, with no requirement for a manually tuned threshold, making ICS-Sniper portable.

We also performed the following two supplementary evaluations of ICS-Sniper’s superperiod estimation.

(i) *Accuracy under uniform packet sizes:* We simulate a setting where VPN encryption equalizes every packet size, thereby removing the size feature and forcing ICS-Sniper to rely on IPT alone. To do this, we restricted ICS-Sniper to only the first two timing-dependent steps of superperiod identification, i.e., $\phi_{overlap}$ and $\phi_{interval}$ (Refer to §IV-B, Step 3). We evaluated this restricted version of ICS-Sniper on all the S-SWaT and H-SWaT configurations listed in Table II. Across all S-SWaT configurations and H-MD-BASE, ICS-Sniper was able to estimate the superperiod with 100% accuracy. For the S-SWaT configurations with low message rates, Step $\phi_{overlap}$ yielded the highest alignment score for the correct superperiod, while for the H-SWaT testbed with a high message rate, Step $\phi_{interval}$ scored the highest. However, ICS-Sniper could not detect the superperiod for H-MD-LOSS1 and H-MD-LOSS2, since under lossy network conditions packet length is the only feature with reliable periodicity. However, in real ICSes and Internet, such a sustained aggressive packet drop is rare.

(ii) *Scalability with the number of gateway PLCs:* To evaluate superperiod detection at scale, we increased the number of gateway PLCs in the S-EN-PCP configuration from 6 to 12 by duplicating the original set of PLCs. This resulted in six PLCs with a 60 s period and six with a 90 s period, with all 12 sharing a single VPN tunnel. We found that ICS-Sniper estimated the superperiod (180 s) with 100% accuracy, despite the message rate (i.e., number of messages transmitted per second between the PLCs and SCADA over the VPN tunnel) being doubled. This robustness stems from ICS-Sniper’s use of three complementary metadata features: overlap region boundaries, IPTs, and packet lengths. Doubling the message rate on the VPN tunnel without lengthening the individual PLC periods obscured the overlap region boundaries, but the periodicity of individual IPT classes and distinctly sized packets from each PLC were retained, which allowed ICS-Sniper to recover the superperiod from the $\phi_{interval}$ and ϕ_{pktlen} steps.

Identifying critical superperiods. During profiling, ICS-Sniper flags a superperiod as critical if it differs from its predecessor under the two heuristics of Step 4 of §IV-B, prioritizing the one with the largest difference when several qualify. Therefore, we measure ICS-Sniper’s effectiveness at identifying critical superperiods by the position of the first true critical superperiod on its priority list. This is an indicator of how many operational cycles ICS-Sniper would require to successfully disrupt a critical state transition. Since ICS-

Detector	Type	Technique	Feature
Inter-arrival time [30] (IaT-mean, IaT-range)	N	Statistical modeling of sample distribution (mean/range)	Interval between consecutive occurrences of the same SCADA-PLC message
Kitsune [29]	N	Ensemble of neural networks for anomaly detection	Temporal statistics of packet sizes, count, and jitter
InvariantRules [26]	P	Invariant rules for sensor/actuator values	Sensor and actuator values
PASAD [27]	P	Singular Spectrum Analysis (SSA)	Time series of individual sensor readings
Histogram, MinMax, SteadyTime [28]	P	Threshold-based statistical modeling trained per sensor/actuator	Properties of process variables (e.g., value range, temporal stability)

TABLE I: ICS attack detectors used in our experiments. N = Network-based, P = Process log-based.

Testbed	Setup	Protocol	PLC Comm. periods (s)	Scan cycle (s)	Message rate (msgs/s)	Network conditions
S-SWaT	S-MD-BASE	Unconnected Modbus TCP	60, 60, 60, 70, 70, 70	60	≈ 40	No added perturbation
	S-EN-BASE	Unconnected ENIP	60, 60, 60, 70, 70, 70	60	≈ 40	No added perturbation
	S-EN-RNC	Unconnected ENIP	60, 60, 60, 70, 70, 70	60	≈ 40	Different day, natural perturbations
	S-EN-PCP	Unconnected ENIP	60, 60, 60, 90, 90, 90	60	≈ 40	No added perturbation
H-SWaT	H-MD-BASE	Connected Modbus TCP	5, 5, 10, 6, 10, 6	0.005	≈ 65	No added perturbation
	H-MD-LOSS1	Connected Modbus TCP	5, 5, 10, 6, 10, 6	0.005	≈ 65	1% packets dropped
	H-MD-LOSS2	Connected Modbus TCP	7, 7, 10, 6, 10, 6	0.005	≈ 65	1% packets dropped

TABLE II: Testbed configurations for ICS-Sniper evaluation. Highlighted cells indicate changes from one row to the next.

Sniper drops packets during only two superperiods per cycle, a cycle whose targeted superperiods contain no critical transition has no effect, forcing it to wait for the next cycle and the next candidate on the list. Fewer required cycles thus mean faster success and greater stealth. In 71% of cases, ICS-Sniper accurately identified the most critical superperiod as its first choice. In the remaining cases, the critical superperiod was ranked as the second most prioritized candidate by ICS-Sniper. Table VI in Appendix E shows the critical state transitions identified by ICS-Sniper and the number of attempts required.

Key insights. (1) While all three event types ($\phi_{overlap}$, $\phi_{interval}$ and ϕ_{pktlen}) used for superperiod identification (§IV-B) were effective under stable network conditions and low message rates, $\phi_{overlap}$ became unreliable under lossy network conditions, high message rates, and very short periods, which produce very short and/or inconsistent overlap regions. However, in these cases $\phi_{interval}$ and ϕ_{pktlen} still identified the superperiods accurately. In H-SWaT, although most payload packets were similarly sized, enough distinct sizes recurred across PLCs and states for ICS-Sniper to recover the superperiod. This confirms that our feature selection and ETA technique are robust across network conditions and PLC period configurations. (2) ICS-Sniper can successfully identify the critical superperiods where, after a critical state transition, the process remains in the new state for an extended duration, or where multiple critical transitions occur in close succession. Both of these scenarios occur in our testbeds.

Summary: ICS-Sniper’s process profiling technique is accurate and robust across network conditions, protocols, message rates, and PLC configurations.

B. Attack Impact

Since ICS-Sniper achieves 100% profiling accuracy across configurations, its attack effectiveness is equally consistent. We demonstrate this on S-EN-BASE and H-MD-BASE.

Experiment. As mentioned in §III-C, ICS-Sniper is process-unaware, hence agnostic of the implications of the state transitions and delayed transitions on the plant. It simply drops packets whenever it predicts an upcoming critical state transition during the active attack stage. We execute one attack per operational cycle and assess its impact by inspecting whether it causes the overall process to deviate from normal behavior, resulting in reduced throughput at the end of the operational cycle, or a safety constraint violation. We study the impact of the following attacks on our testbeds:

Attacks on S-EN-BASE. ICS-Sniper drops the most critical superperiod (occurring within the first 20 minutes of the operational cycle) and the second most critical superperiod (occurring within the final 30 minutes). We refer to these attacks as *Process Delay Attack* and *UF-Depletion Attack*, respectively, based on the effect that the attacks are expected to have on the underlying sub-process. **Attack on H-MD-BASE.** ICS-Sniper drops the most critical superperiod (occurring within the first minute of the operational cycle). This is also a variant of the process delay attack.

Process Delay Attack. In this attack, ICS-Sniper dropped all payload packets during the first two superperiods of the process, i.e., for 14 minutes in S-EN-BASE, and for 1 minute in H-MD-BASE. Within this time, ICS-Sniper dropped $\approx 4.7\%$ and $\approx 2.5\%$ of the total number of packets in S-EN-BASE and H-MD-BASE, respectively, exchanged between the site and the SCADA over the complete operational cycle. During this initial part of the operational cycle, the SCADA

issues control commands to initialize PLC operations, while multiple sub-processes synchronize with one another. Packet drops within these superperiods *disrupted five critical state transitions* (shown in Table VI), stalling the operations of the sub-processes and causing a delayed process start.

Impact. The delay caused by this attack had a cascading effect across the sub-processes, ultimately *postponing the start of clean-water pumping by ≈ 15 minutes* in S-EN-BASE and by *2 minutes* in H-MD-BASE. In H-MD-BASE, the impact cascaded further, eventually requiring 3 extra minutes of plant operation to meet its requirements. An average small-scale water treatment plant that pumps 100 gallons of water per minute would have a deficit of 3000 gallons and 300 gallons for S-EN-BASE and H-MD-BASE, respectively.

UF-Depletion Attack. In this attack, ICS-Sniper dropped all payload packets exchanged between the site and the SCADA during the 31st and 32nd superperiods of the process, i.e., for ≈ 14 minutes. Within this time, ICS-Sniper dropped $\approx 4.1\%$ of the total number of packets exchanged between the site and the SCADA over the complete operational cycle. The attack stalled a critical state transition (State 13 $\rightarrow$ State 14) by disrupting synchronization between the Ultrafiltration (P3) and Backwash (P6) sub-processes. Together, they execute the ultrafiltration–backwash cycle: during backwash, P3 opens a valve to reverse flow through the UF membranes while P6 activates a pump to redirect the backwash stream to the feed tank. When P6’s pump stops, P3’s valve must close immediately to prevent drainage. By delaying the control message from P6 to P3 signaling the pump stoppage, the attack broke this tight coordination.

Impact. This attack caused the backwash valve in P3 to remain open for an *additional 9 minutes and 11 seconds* after the pump in P6 stopped. This would likely result in substantial treated water loss: P6 would fail to return excess backwash water to the feed tank, and P2 would be unable to pump chemically treated water to P3 for ultrafiltration, stalling the ultrafiltration cycle and reducing overall process throughput.

Summary: All active attacks caused deviations from ideal behavior, causing degradation of plant throughput. In practice, this would incur additional operating hours or unscheduled maintenance, impose a financial burden on the plant operator, impact downstream processes, disrupt water supply to critical public services, or lower the water pressure.

C. Stealthiness: Detecting ICS-Sniper

Training the detectors. We equipped our testbed with an ensemble of eight anomaly detectors, as described in §V-C. They were trained with data from two complete operational cycles. The network-based detectors were trained with unencrypted traffic captured at the SCADA and the process log-based detectors were trained with sensor and actuator values collected from the PLCs, and logged by the SCADA. S-SWaT logs every 10 seconds, while H-SWaT, with a shorter scan cycle period, logs every second. We fine-tuned the parameters

(e.g., window size) in the detectors such that they raise the minimum number of false alarms for benign SCADA traffic.

Detector success. For a detector to successfully identify an ICS-Sniper attack, it must detect anomalous behavior *and* attribute it to the site-to-SCADA network path. This dual requirement motivates the use of both network-based and process log-based detectors. Network-based detectors alone cannot confirm process impact, as network traffic anomalies can be benign. Similarly, process log-based detectors alone cannot localize an anomaly to the network path, as sensor reading anomalies might arise due to attacks on PLCs and sensors at the site. Without both, plant operators cannot implement an effective mitigation. For instance, if only a process log-based detector raises an alert and the operator probes the site-to-SCADA link for liveness, ICS-Sniper would not drop probe packets unless they satisfy the ICS protocol-specific size constraints, leaving the operator unaware that the communication path is compromised. Furthermore, an effective detector must raise an alert (i.e., detect an anomaly) as early as possible during the attack and keep raising alerts throughout the attack duration.

We measure detector success using two metrics. (1) **Detection lag:** the time difference between the attack start time and the time of the first alert that is related to the attack event, and (2) **Alert overlap:** the time overlap between the alerts raised and the active attack window. A good detector should have a low detection lag and a high overlap.

Evaluation strategy. To evaluate ICS-Sniper’s stealthiness across various generalization parameters, we measure the detector success metrics over 5 scenarios: ① Process delay attack on S-EN-BASE, ② UF-Depletion attack on S-EN-BASE, ③ Process delay attack on H-MD-BASE, ④ Process delay attack on H-MD-LOSS1, and ⑤ Process delay attack on H-MD-LOSS2.

Results. Table III summarizes the performance of each detector type across these scenarios. Figures 9 and 10 in Appendix F show the efficacy of the individual and ensemble alerts of network-based and process log-based anomaly detectors, respectively, for Scenario ②. Figures 11 and 12 in Appendix F show the same for Scenario ③. In summary, while for every scenario at least one detector raised an alert at some point during the operational cycle (the ANY ensemble raised an alert), no scenario resulted in alerts from both detector types during the active attack stage (the ALL ensemble never raised an alert). For 2 out of 5 scenarios, no detector raised an alert during the active attack stage. Across all scenarios, the process underwent performance degradation before both types of detectors raised alerts. The impact was process delay ranging from 2 mins (Scenario ③) to 15 mins (Scenario ①).

Key insights. (1) No network-based detector raised an alert until the attack had concluded. This is due to their fundamental design characteristic: their anomaly detection logic is event-driven, triggering only upon packet arrival, leaving even prolonged silent periods on the communication link undetected. This design suits volumetric DoS attacks, where it enables real-time response under high packet rates, but leaves the

Scenario	Dur	Det	Rel	Lag	Overlap
①	840	■ Kitsune	Yes	854.7	0
		★ InvariantRules	Yes	867.7	0
②	840	■ Kitsune, IaT-mean, IaT-range	Yes	850.8	0
		★ SteadyTime	No	–	–
③	60	■ Kitsune	Yes	142.7	0
		★ InvariantRules	Yes	0.4	59.6
④	60	■ Kitsune	Yes	142.7	0
		★ InvariantRules	Yes	0.4	59.6
⑤	420	■ Kitsune, IaT-mean	Yes	≈515	0
		★ Histogram	Yes	194.5	225.5

TABLE III: Performance of attack detectors on ICS-Sniper.

■: network-based detectors, ★: process log-based detectors. **Dur**: Attack duration (s), **Det**: detector with smallest detection lag and/or longest alert overlap, **Rel**: relevance of alert to attack, **Lag**: Detection lag (s), **Overlap**: Alert overlap (s).

detectors blind to blackhole attacks like ICS-Sniper. These findings highlight the need for a new detection approach for targeted blackhole attacks on ICS networks. Detecting such attacks would instead require periodically polling the communication link, synchronized to PLC periodicities and network conditions, which is a non-trivial research challenge.

(2) Unlike H-SWaT, no process log-based detector raised alerts in S-SWaT. This is due to S-SWaT’s longer scan cycle and 10× slower logging rate, which reduce observability of process value drifts and prevent detectors from triggering.

(3) Detector performance is best under stable network conditions (< 1% packet drops) and short PLC communication periods (Scenarios ③ and ④). This is because longer periods in a lossy network slow the message rate, making deviations harder to detect.

Summary: No detector could effectively detect ICS-Sniper during its active attack window across all generalization parameters.

VII. DISCUSSION

Impact of sporadic manual commands and out-of-routine state changes. These factors do not mislead ICS-Sniper, because its profiling stage repeats the traffic-capture step over multiple operational cycles until a consistent per-cycle packet count recurs, then selects a representative sequence. Therefore, cycles perturbed by such events are discarded and never drive prediction. These events might raise ICS-Sniper’s monitoring cost, but a nation-state actor may have the resources to stealthily monitor the ICS for a long time before attacking.

Limitation of ICS-Sniper. ICS-Sniper’s efficacy relies on the difference in IPTs between a batch-processing ICS’s operational and idle phases. Continuous-processing ICSes lack distinct operational and idle phases, which may render ICS-Sniper ineffective against them. However, many safety-critical ICSes are batch-processing systems [38], [39], against which ICS-Sniper remains effective.

Countermeasures. ICS-Sniper would not succeed if no on-path network device or service had an exploitable vulnerability. However, securing every network device outside the ICS perimeter is highly challenging, as it requires collaboration with ISPs and a deep understanding of the impact of packet drops at different points during the operational cycle. Therefore, we consider other countermeasures.

Traffic shaping. ICS-Sniper relies on periodic burst patterns in PLC traffic to predict state transitions. Therefore, a principled approach to mitigating the attacks would require *shaping* the traffic to obfuscate the periodic burst patterns. Traffic shaping can be implemented at either the application or network layer; however, PLC periodicities and burst durations are typically tuned for optimal performance. Designing a shaping strategy that ensures security without sacrificing performance remains an open research challenge.

Redundant network paths. ICSes could route their OT traffic over multiple Internet paths, e.g., using source routing [49].

VIII. RELATED WORK

Encrypted Traffic Analysis-based Attacks. Encrypted Traffic Analysis (ETA) has been used for inferring users’ website visits [50] and IoT device activity [51], [52], where an adversary first profiles the traffic by interacting with the target as a legitimate user. However, ICS environments lack a public-facing interface, making such interactions, and thus detailed behavioral profiling, infeasible for an external adversary. Existing work has only coarsely distinguished encrypted ICS control traffic from general Internet traffic [53], without conducting fine-grained analysis of ICS message criticality.

Existing ICS DoS Attacks. DoS attacks on ICSes can be classified based on the adversary’s vantage point. (i) *Data modification attacks* involve infiltrating ICS networks to modify or drop time-sensitive sensor readings, disrupting critical operations [22], [54]. (ii) *Traffic flow modification attacks* are carried out by adversaries who cannot infiltrate the ICS network and thus cannot observe packet contents to assess their importance. These adversaries may use two main tactics: (i) *Volumetric DoS attacks* that gradually overload a service with illegitimate requests, preventing it from processing legitimate ones [55], and (ii) *Blackhole attacks* that disable a target service by dropping all traffic sent to it [56], [57], [58], [59].

DoS detection techniques. Apart from the ICS DoS detection techniques in §V-C, there are blackhole attack detectors and conventional cloud-based DoS detectors. Blackhole attack detectors [56], [58], [59] focus on anomalies in route discovery, but struggle with attacks that only drop a few packets. Conventional cloud defenses target ARP poisoning, payload tampering, and high-volume floods through implementing static ARP, endpoint security, and high-volume-flow detection [44], [45], [46]. ICS-Sniper introduces no malicious ARP, alters no payload, and generates no extra traffic, and only selectively drops a few packets. Hence, while these defenses are effective in securing the web services that dominate cloud traffic, they are not designed to detect ICS-Sniper.

IX. CONCLUSION

We present *ICS-Sniper*, the first targeted blackhole DoS attack technique for modern batch-processing Industrial Control Systems (ICS) that host the SCADA on the cloud. ICS-Sniper leverages the periodic and bursty nature of ICS communications and the correlation between packet metadata and ICS states to infer underlying state transitions and critical messages. It can attack an ICS from a compromised ISP device outside the ICS perimeter, causing substantial damage while remaining undetected. Our case study on a water treatment ICS testbed demonstrates that ICS-Sniper can potentially cause loss of productivity of the ICS. As cloud-hosted SCADA becomes a necessity, our findings underscore the need for stronger protections in modern ICSes beyond the NIST recommendations. To mitigate ICS-Sniper, we recommend strong countermeasures based on traffic shaping and redundancy in routing.

Future work will focus on expanding ICS-Sniper to a larger class of ICSes, including continuous processing ones. We will also explore mitigation strategies against the attacks.

ACKNOWLEDGEMENTS

This research was supported in part by the National Cybersecurity Consortium of Canada (NCC), the Natural Sciences and Engineering Research Council of Canada (NSERC) and the Digital Research Alliance of Canada (DRAC).

X. ETHICS CONSIDERATION

We have made our best efforts to conduct our research ethically. We highlight our considerations at specific research steps.

Stakeholders. This research was conducted following a stakeholder-based ethics analysis. We identified several key stakeholders who could be impacted by our research process and its publication:

- The Research Team: The authors on this paper.
- The ICS: The plant being targeted in the attack. (i) Our reconstruction of the superperiodic structure for the SWaT plant at SUTD might raise concerns about threats to the physical plant. However, the original plant's OT system is not connected to the Internet. Therefore, an adversary cannot leverage our work to attack the physical plant. (ii) Our attacks shown in §VI have been performed only on an emulated testbed that contains no physical components.
- Users: The end users served by the plant.
- The Security Community: Our peers who build upon and validate security research.
- Malicious Actors: Those who might attempt to exploit the vulnerabilities we discovered.

Risk vs. beneficence of our research. We have considered the Menlo Report principles [60], particularly “Beneficence” [61], balancing research benefits with privacy protection. While our research introduces some risk that adversaries might launch the attacks exposed in this work, we believe that the benefits of revealing the information for building secure ICS solutions outweigh the potential threat.

Other considerations. Our work does not involve human subjects or personal data. All experiments were conducted on a local setup or a separate cloud setup that did not involve or affect other users.

XI. LLM USAGE CONSIDERATIONS

No new text or image in this paper has been generated using LLMs. However, we have used LLM (Claude Sonnet 4.6) to check the grammar and consistency of the text.

REFERENCES

- [1] European Network and Information Security Agency, “Industrial Control Systems Security: Recommendations for Europe & Member States,” URL: <https://www.enisa.europa.eu/news/enisa-news/industrial-control-systems-security-recommendations-for-europe-member-states>, Last accessed: May 6, 2026.
- [2] The Register, “Cyberpunks mess with Canada’s water, energy, and farm systems,” URL: <https://www.theregister.com/security/2025/10/30/cyberpunks-mess-with-canadas-water-energy-farm-systems/602770>, Last accessed: May 6, 2026.
- [3] CNN, “Russia-linked hacking group suspected of carrying out cyberattack on Texas water facility,” URL: <https://www.cnn.com/2024/04/17/politics/russia-hacking-group-suspected-texas-water-cyberattack/index.html>, Last accessed: May 6, 2026.
- [4] Amazon AWS, “Improve your industrial operations with cloud-based SCADA systems,” URL: <https://aws.amazon.com/blogs/industries/improve-your-industrial-operations-with-cloud-scada/>, Last accessed: May 6, 2026.
- [5] VTScada, “Using Cloud-Based SCADA to Provide Advanced Cost-Effective Solutions to Small and Midsize Users,” URL: <https://www.vtscada.com/using-cloud-based-scada-to-provide-advanced-cost-effective-solutions-to-small-and-midsize-users/>, Last accessed: May 6, 2026.
- [6] K. Stouffer, V. Pillitteri, M. Abrams, and A. Hahn, “Guide to Operational Technology (OT) Security,” URL: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-82r3.pdf>, Last accessed: May 6, 2026.
- [7] Amazon AWS, “Security best practices for your VPC,” URL: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-best-practices.html>, Last accessed: May 6, 2026.
- [8] S. E. Coull, “Traffic analysis,” in *Encyclopedia of Cryptography, Security and Privacy*. Springer, 2025, pp. 2638–2641.
- [9] S. Zucker, “Detection of periodicity based on serial dependence of phase-folded data,” *Monthly Notices of the Royal Astronomical Society*, vol. 449, no. 3, pp. 2723–2733, 2015.
- [10] SUTD iTrust, “Secure Water Treatment (SWaT) Test-bed, SUTD, Singapore,” URL: <https://www.sutd.edu.sg/itrust/swat/>, Last accessed: May 6, 2026.
- [11] OpenVPN, “OpenVPN Protocol,” URL: <https://openvpn.net/community-docs/openvpn-protocol.html>, Last accessed: May 6, 2026.
- [12] K. Wolsing, D. Kus, E. Wagner, J. Pennekamp, K. Wehrle, and M. Henze, “One IDS is not enough! Exploring ensemble learning for industrial intrusion detection,” in *Computer Security – ESORICS 2023*. Springer, 2024, pp. 102–122.
- [13] SANS Institute, “SANS 2024 State of ICS/OT Cybersecurity,” URL: <https://sansorg.egnyte.com/dl/oq4USL3qoA>, Last accessed: May 6, 2026.
- [14] Data Command, “SCADA Replacement,” URL: <https://www.data-command.com/case-studies/scada-interface/>, Last accessed: May 6, 2026.
- [15] Yokogawa, “SCADA as a Service,” URL: https://web-material3.yokogawa.com/SCADA_as_a_Service.au.pdf, Last accessed: May 6, 2026.
- [16] Modbus, “MODBUS APPLICATION PROTOCOL SPECIFICATION,” URL: <https://www.modbus.org/file/secure/modbusprotocollspecification.pdf>, Last accessed: May 6, 2026.
- [17] Real Time Automation, “EtherNet/IP,” URL: <https://www.rtautomation.com/technologies/ethernetip/>, Last accessed: May 6, 2026.

- [18] M. H. Collantes and A. L. Padilla, "Protocols and network security in ICS infrastructures," *INCIBE Technology Report*, 2015, URL: https://www.incibe.es/sites/default/files/contenidos/guias/doc/incibe_protocol_net_security_ics.pdf, Last accessed: May 6, 2026.
- [19] Modbus, "MODBUS MESSAGING ON TCP/IP IMPLEMENTATION GUIDE," URL: https://www.modbus.org/file/secure_messagingimplementationguide.pdf, Last accessed: May 6, 2026.
- [20] P. Brooks, "EtherNet/IP: Industrial Protocol White Paper," URL: https://literature.rockwellautomation.com/idc/groups/literature/documents/wp/enet-wp001_-en-p.pdf, Last accessed: May 6, 2026.
- [21] Acromag, "Introduction to MODBUS TCP/IP," URL: https://www.prosoft-technology.com/kb/assets/intro_modbustcp.pdf, Last accessed: May 6, 2026.
- [22] M. Krotofil, A. A. Cárdenas, B. Manning, and J. Larsen, "CPS: Driving cyber-physical systems to unsafe operating conditions by timing DoS attacks on sensor signals," in *ACSAC*, 2014, pp. 146–155.
- [23] Schneider Electric USA, Inc., "MODBUS/TCP Security," URL: <https://blog.se.com/industry/machine-and-process-management/2018/08/30/modbus-security-new-protocol-to-improve-control-system-security/>, Last accessed: May 6, 2026.
- [24] J. S. Rinaldi, "EtherNet/IP and Transport Layer Security," URL: <https://www.rtautomation.com/rtas-blog/ethernet-ip-and-transport-layer-security/>, Last accessed: May 6, 2026.
- [25] Center for the Protection of National Infrastructure, "Configuring and Managing Remote Access for Industrial Control Systems," URL: https://www.cisa.gov/sites/default/files/2023-01/RP_Managing_Remote_Access_S508NC.pdf, Last accessed: May 6, 2026.
- [26] F. Cheng, V. R. Palleti, A. Mathur, and D. Chana, "A systematic framework to generate invariants for anomaly detection in industrial control systems," in *NDSS Symposium*, 2019.
- [27] W. Aoudi, M. Iturbe, and M. Almgren, "Truth will out: departure-based process-level detection of stealthy attacks on control systems," in *ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 817–831.
- [28] K. Wolsing, L. Thiemt, C. van Sloun, E. Wagner, K. Wehrle, and M. Henze, "Can industrial intrusion detection be simple?" in *European Symposium on Research in Computer Security*. Springer, 2022, pp. 574–594.
- [29] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *NDSS Symposium*, 2018.
- [30] C.-Y. Lin, S. Nadjm-Tehrani, and M. Asplund, "Timing-based anomaly detection in SCADA networks," in *Critical Information Infrastructures Security*. Springer, 2018, pp. 48–59.
- [31] Allen Bradley, "Allen Bradley Programming Manual: Logix5000 Controllers Tasks, Programs, and Routines," URL: https://literature.rockwellautomation.com/idc/groups/literature/documents/pm/1756-pm005_-en-p.pdf, Last accessed: May 6, 2026.
- [32] CSO Online, "Juniper Router Attack," URL: <https://www.csoonline.com/article/3844122/chinese-cyberespionage-group-deploys-custom-backdoors-on-juniper-routers.html>, Last accessed: May 6, 2026.
- [33] Cisco, "Multiple Vulnerabilities in Cisco IOS XE Software Web UI Feature," URL: <https://sec.cloudapps.cisco.com/security/center/content/CiscoSecurityAdvisory/cisco-sa-iosxe-webui-privesc-j22SaA4z>, Last accessed: May 6, 2026.
- [34] —, "Cisco IOS and IOS XE Software Smart Install Remote Code Execution Vulnerability," URL: <https://www.cisco.com/c/en/us/support/docs/csa/cisco-sa-20180328-smi2.html>, Last accessed: May 6, 2026.
- [35] MITRE, "Junos OS Evolved: Shell Injection vulnerability in the gNOI server," URL: <https://www.cve.org/CVERecord?id=CVE-2023-28983>, Last accessed: May 6, 2026.
- [36] D. Xue, R. Ramesh, A. Jain, M. Kallitsis, J. A. Halderman, J. R. Crandall, and R. Ensafi, "OpenVPN is open to VPN fingerprinting," *Communications of the ACM*, vol. 68, no. 1, pp. 79–87, 2025.
- [37] Y. S. Razooqi and A. Pekar, "VPN Traffic Analysis: A Survey on Detection and Application Identification," *IEEE Access*, vol. 13, pp. 132 830–132 848, 2025.
- [38] JMark Systems, "Batch vs. Continuous Wastewater Treatment Systems," URL: <https://www.jmarksystems.com/batch-vs-continuous-wastewater-treatment-systems/>, Last accessed: May 6, 2026.
- [39] GE Vernova, "Batch Processing Plants," URL: <https://www.gevernova.com/software/blog/continuous-batch-manufacturing-differences#manufacturing-models>, Last accessed: May 6, 2026.
- [40] iGRID, "IEC 61850 MMS," URL: <https://www.igrid-td.com/smartguide/iec61850/iec-61850-mms/>, Last accessed: May 6, 2026.
- [41] SUTD iTrust, "Electric Power and Intelligent Control (EPIC)," URL: <https://www.sutd.edu.sg/itrust/itrust-labs/datasets/dataset-characteristics/epic/>, Last accessed: May 6, 2026.
- [42] Y. Chen, "SWaT Logic," URL: https://github.com/yuqiChen94/Swat_Simulator, Last accessed: May 6, 2026.
- [43] Wireshark, "tshark: Terminal-based Wireshark," URL: https://www.wireshark.org/docs/wsug_html_chunked/AppToolstshark.html, Last accessed: May 6, 2026.
- [44] Amazon, "AWS Shield," URL: <https://aws.amazon.com/shield/>, Last accessed: May 6, 2026.
- [45] M. Reece, N. Rastogi, T. Lander, J. Dykstra, S. Mittal, and A. Sampson, "Defending multi-cloud applications against man-in-the-middle attacks," in *Proceedings of the 29th ACM Symposium on Access Control Models and Technologies*, 2024, pp. 47–52.
- [46] S. K. Fayaz, Y. Tobioka, V. Sekar, and M. Bailey, "Bohatei: Flexible and elastic DDoS defense," in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 817–832.
- [47] J. Goh, S. Adepu, K. N. Junejo, and A. P. Mathur, "A dataset to support research in the design of secure water treatment systems," in *International Workshop on Critical Information Infrastructures Security*. Springer, 2017, pp. 88–99.
- [48] C. M. Ahmed, V. R. Palleti, and A. P. Mathur, "WADI: a water distribution testbed for research in the design of secure cyber physical systems," in *3rd International Workshop on Cyber-Physical Systems for Smart Water Networks (CySWATER)*, 2017, pp. 25–28.
- [49] D. Barrera, L. Chuat, A. Perrig, R. M. Reischuk, and P. Szalachowski, "The SCION Internet Architecture," *Communications of the ACM*, vol. 60, no. 6, pp. 56–65, 2017.
- [50] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, "Effective attacks and provable defenses for website fingerprinting," in *USENIX Security*, 2014, pp. 143–157.
- [51] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Computing Surveys (CSUR)*, vol. 54, no. 6, pp. 1–35, 2021.
- [52] N. Msadek, R. Soua, and T. Engel, "IoT device fingerprinting: Machine learning based encrypted traffic analysis," in *Wireless Communications and Networking Conference (WCNC)*. IEEE, 2019, pp. 1–8.
- [53] G. Barbieri, M. Conti, N. O. Tippenhauer, and F. Turrin, "Assessing the Use of Insecure ICS Protocols via IXP Network Traffic Analysis," in *International Conference on Computer Communications and Networks (ICCCN)*. IEEE, 2021, pp. 1–9.
- [54] G. K. Ndonga and R. Sadre, "Exploiting the Temporal Behavior of State Transitions for Intrusion Detection in ICS/SCADA," *IEEE Access*, vol. 10, pp. 111 171–111 187, 2022.
- [55] O. Gasser, Q. Scheitle, B. Rudolph, C. Denis, N. Schricker, and G. Carle, "The amplification threat posed by publicly reachable BACnet devices," *Journal of Cyber Security and Mobility*, vol. 6, no. 1, pp. 77–104, 2017.
- [56] R. R. Mahmood and A. Khan, "A survey on detecting black hole attack in AODV-based mobile ad hoc networks," in *International Symposium on High Capacity Optical Networks and Enabling Technologies*. IEEE, 2007, pp. 1–6.
- [57] T. N. D. Pham and C. K. Yeo, "Detecting colluding blackhole and greyhole attacks in delay tolerant networks," *IEEE Transactions on Mobile Computing*, vol. 15, no. 5, pp. 1116–1129, 2015.
- [58] D. Airehrour, J. Gutierrez, and S. K. Ray, "Securing RPL routing protocol from blackhole attacks using a trust-based mechanism," in *International Telecommunication Networks and Applications Conference (ITNAC)*. IEEE, 2016, pp. 115–120.
- [59] N. Abosata, S. Al-Rubaye, G. Inalhan, and C. Emmanouilidis, "Internet of things for system integrity: A comprehensive survey on security, attacks and countermeasures for industrial applications," *Sensors*, vol. 21, no. 11, p. 3654, 2021.
- [60] E. Kenneally and D. Dittrich, "The Menlo Report: Ethical Principles Guiding Information and Communication Technology research," *Available at SSRN 2445102*, 2012.
- [61] T. Kohno, Y. Acar, and W. Loh, "Ethical frameworks and computer security trolley problems: Foundations for conversations," in *USENIX Security*, 2023, pp. 5145–5162.

APPENDIX

A. Sample control algorithm

```

1 PROGRAM P1_Control
2 //Initializing parameters
3 VAR
4     // Overall plant status
5     plantOn: BOOL := False;
6     // Sub-process state
7     p1State: INT := 1;
8     // device configurations
9     pmp1Status: BOOL := False;
10    vlv1Status: BOOL := False;
11    vlv2Status: BOOL := False;
12    //sensor values
13    lvl1Val: INT := Low;
14 END_VAR
15 // Main block
16 IF p1State==1 THEN
17     // Obtain remote input from SCADA
18     GETVAL(plantOn)
19     GETVAL(vlv2Status)
20     // Control logic
21     IF plantOn THEN
22         IF vlv2Status==True AND lvl1Val>Low THEN
23             pmp1Status := True;
24         END_IF
25         IF lvl1Val<High THEN
26             vlv1Status := True;
27         END_IF
28         IF lvl1Val==High THEN
29             vlv1Status := False;
30         END_IF
31         IF vlv2Status==False THEN
32             // State transition
33             p1State := 2;
34         END_IF
35     END_IF
36     // Send data updates to SCADA
37     SETVAL(lvl1Val)
38     SETVAL(pmp1Status)
39     SETVAL(vlv1Status)
40 END_IF
41 IF p1State==2 THEN
42     // Obtain remote inputs from SCADA
43     GETVAL(plantOn)
44     // Control logic
45     pmp1Status := False
46     IF !plantOn OR lvl1Val==High THEN
47         vlv1Status := False;
48         // State transition
49         p1State := 1;
50     ELSE
51         vlv1Status := True;
52     END_IF
53     // Send data updates to SCADA
54     SETVAL(lvl1Val)
55     SETVAL(pmp1Status)
56     SETVAL(vlv1Status)
57 END_IF

```

```

1 PROGRAM P2_Control
2 //Initializing devices
3 VAR
4     // Overall plant status
5     plantOn: BOOL := False;
6     // Sub-process states
7     p2State: INT := 1;
8     // device configurations
9     pmp2Status: BOOL := False;
10    vlv2Status: BOOL := False;
11    //sensor values
12    lvl2Val: INT := Low;
13    flw2Val: INT := 0;
14 END_VAR
15 // Main block
16 IF p2State==1 THEN
17     // Obtain remote input from SCADA
18     GETVAL(plantOn)
19     // Control logic
20     IF plantOn THEN
21         vlv2Status := True
22         IF lvl2Val>Low AND lvl2Val<High AND flw2Val<W THEN
23             pmp2Status := True;
24         END_IF
25         IF lvl2Val==High THEN
26             // State transition
27             p2State := 2;
28         END_IF
29     ELSE
30         pmp2Status := False;
31         vlv2Status := False;
32     END_IF
33     // Send data updates to SCADA
34     SETVAL(lvl2Val)
35     SETVAL(pmp2Status)
36     SETVAL(vlv2Status)
37     SETVAL(flw2Val)
38 END_IF
39 IF p2State==2 THEN
40     // Obtain remote inputs from SCADA
41     GETVAL(plantOn)
42     // Control logic
43     vlv2Status := False;
44     IF flw2Val<W THEN
45         pmp2Status := True;
46     END_IF
47     IF flw2Val==W OR !plantOn THEN
48         pmp2Status := False;
49         vlv2Status := False;
50         // State transition
51         p2State := 1;
52     END_IF
53     // Send data updates to SCADA
54     SETVAL(lvl2Val)
55     SETVAL(pmp2Status)
56     SETVAL(vlv2Status)
57     SETVAL(flw2Val)
58 END_IF

```

Fig. 6: Control algorithms for a sample ICS described in Section III.

B. Corner case analysis of the profiling stage of ICS-Sniper

ICS-Sniper is likely to mispredict state transitions or fail to identify the superperiod in the following cases.

1) ICS-Sniper infers state transitions from changes in traffic burst patterns, but it cannot distinguish whether a transition was triggered by a local sensor input or by remote sub-process input. If all the transitions in the identified superperiods are

local, then dropping packets will not affect the process, even though the transitions are detected. In such cases, the attack would have a limited impact.

2) Within a superperiod, if multiple sub-processes undergo state transitions such that the aggregate traffic metadata before and after the transitions remains the same, ICS-Sniper would not detect the transitions. While possible, such scenarios would be unusual and would not likely be frequent in practice.

3) If all sub-processes undergo very frequent state transitions, i.e., a large number of per-PLC state transitions inside each superperiod and in every superperiod over the length of the operational cycle, it would be challenging for ICS-Sniper to identify a dominant period.

4) We know that the superperiod L is the LCM of the periods of the constituent PLCs. Theoretically, L can grow very large when PLC periods include a large number of mutually prime factors. If L exceeds one-third of the length of the operational cycle, ICS-Sniper cannot observe repetition of a full superperiod, and therefore cannot compute it. However, in practice, PLC communication cycles are in the order of seconds while operational cycles span several hours, so this scenario is highly unlikely.

C. Comparison with volumetric blackhole attacks

Individual PLC periods are on the order of seconds, far shorter than the operational cycle, which may span hours to a day. The superperiods are therefore also on the order of seconds to a few minutes. Dropping packets in a targeted way within just two small windows thus makes ICS-Sniper stealthier than a conventional volumetric blackhole attack. We compare the stealth of ICS-Sniper with a non-targeted volumetric blackhole attack in terms of the number of packets they drop.

Number of packets dropped by ICS-Sniper. On detecting a burst-pattern change in superperiod x , ICS-Sniper drops all payload packets in superperiods $(x-1)$ and x . The total number of dropped packets is

$$Drops_{ICS-Sniper} = f\% \text{ of } \sum_{i=1}^n \sum_{j=\frac{L(x-2)}{P_i}+1}^{\frac{Lx}{P_i}} pkt_{ij} \quad (1)$$

where n is the number of PLCs, L is the superperiod, P_i is the period of $s(i)$, and pkt_{ij} is the number of packets of $s(i)$ in its j^{th} period, f indicates the percentage of application payload packets within the time window. $f=20$ for unconnected protocol variants, and $f=100$ for connected protocol variants.

Worst-case comparison with a naive adversary. A naive adversary not equipped with ICS-Sniper knows neither L nor the payload packet sizes, and therefore begins dropping packets from the start of the operational cycle, and continues until it drops the first critical packet. If the first critical packet occurs in superperiod x , the *minimum* number of packets it drops (the adversary can drop any number of packets even after that) is

$$Drops_{naive} = \sum_{i=1}^n \sum_{j=1}^{\frac{L(x-1)}{P_i}} pkt_{ij} + c, \quad (2)$$

where c is the number of packets in the x^{th} superperiod until the critical packet. Comparing equations (1) and (2), we see that, for unconnected protocol variants ICS-Sniper drops at least 80% fewer packets than a naive volumetric blackhole attacker. In the connected variant, the two would

drop the same number of packets only when the second superperiod is critical, and the volumetric attack stops after the second superperiod. The stealth of ICS-Sniper becomes more significant as the value of x increases, i.e., for state changes at later stages in the operational cycle, because prolonged indiscriminate packet drops by a naive adversary could be detected with network polling packets used for liveness checks. ICS-Sniper would not drop these polling packets.

Best-case comparison with a naive adversary. In the best-case scenario, the naive adversary picks a random start time and duration and, by chance, drops only the critical packets. Without knowledge of L , it would have infinitely many choices over the entire round of operation. On the other hand, if an ICS-Sniper-equipped adversary wants to pick a random start time and duration for packet drops, they would do so only within the $(x-1)^{th}$ and x^{th} superperiods. Therefore, the naive adversary has a much lower success probability than that of ICS-Sniper due to ICS-Sniper's much smaller search window.

D. Supplementary information about the SWaT testbed

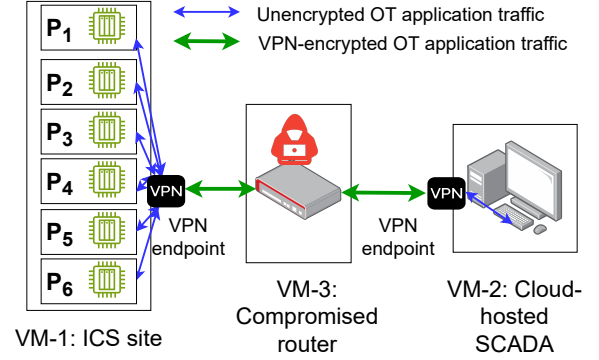


Fig. 7: A schematic diagram of our SWaT testbed

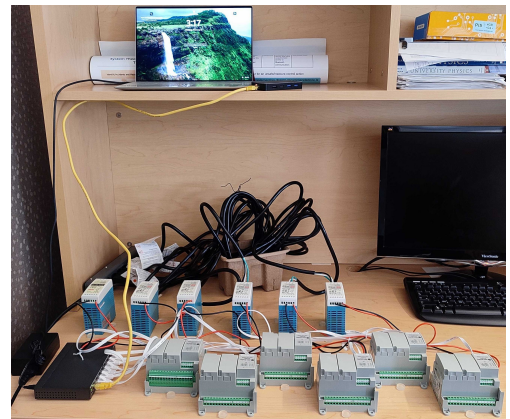


Fig. 8: The site network of our H-SWaT testbed. The six PLCs represent the six sub-processes. The laptop acts as a gateway router and also runs the physical process simulation. The compromised router with ICS-Sniper code and the SCADA are deployed on two different EC2 VMs.

Sub-process	# states	R	# critical msgs	# non-critical msgs
P1	3	2	3	41
P2	2	2	2	100
P3	20	24	4	60
P4	6	1	1	80
P5	21	21	3	60
P6	2	2	3	69

TABLE IV: Information about the state transition model of the SWaT testbed. **R**: # of remotely triggered state transitions.

E. Superperiod identification by ICS-Sniper

Setup	Actual SP (s)	Estimated SP (s)	Accuracy
S-MD-BASE	420	420	100%
S-EN-BASE	420	420	100%
S-EN-RNC	420	420	100%
S-EN-PCP	180	180	100%
H-MD-BASE	30	30	100%
H-MD-LOSS1	30	30	100%
H-MD-LOSS2	210	210	100%

TABLE V: Superperiod (SP) identification accuracy of ICS-Sniper under various testbed configurations.

Setup	# super-periods	# attempts	# critical transitions
S-MD-BASE	33	1	5 (PLC1: 1→2, PLC2: 1→2, PLC3: 1→2, PLC5: 1→3, PLC6: 1→2)
S-EN-BASE	33	2	Same as above
S-EN-RNC	32	2	Same as above
S-EN-PCP	69	1	Same as above
H-MD-BASE	239	1	Same as above
H-MD-LOSS1	239	1	Same as above
H-MD-LOSS2	34	1	Same as above

TABLE VI: Critical superperiods, as identified by ICS-Sniper. Column 3 indicates the number of attempts required to drop the first critical superperiod.

F. Alerts generated by anomaly detectors

Throughout this subsection, red markers denote alerts outside the active attack window, whereas black markers denote alerts overlapping with the active attack window.

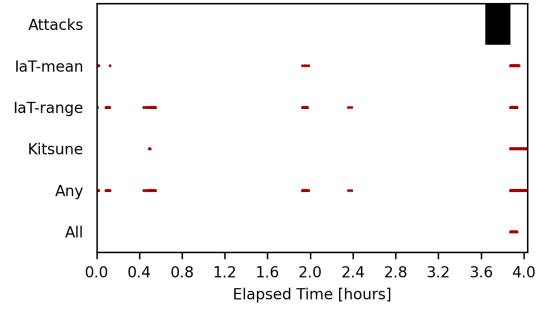


Fig. 9: Alerts generated by the network-based anomaly detectors over time for Scenario (2).

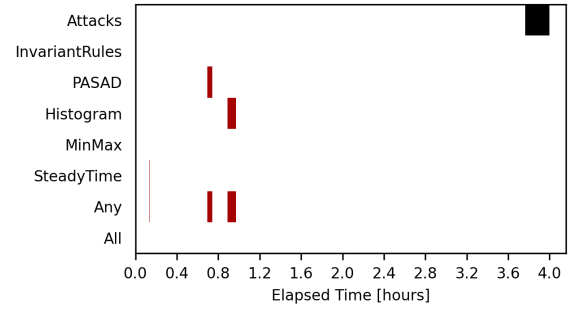


Fig. 10: Alerts generated by the process log-based anomaly detectors over time for Scenario (2).

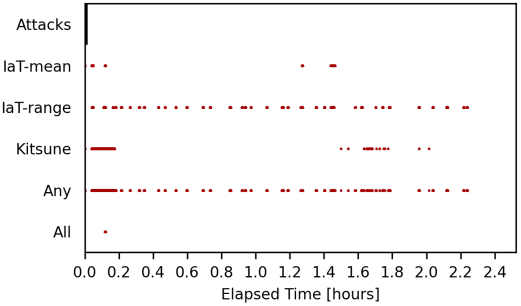


Fig. 11: Alerts generated by the network-based anomaly detectors over time for Scenario (3).

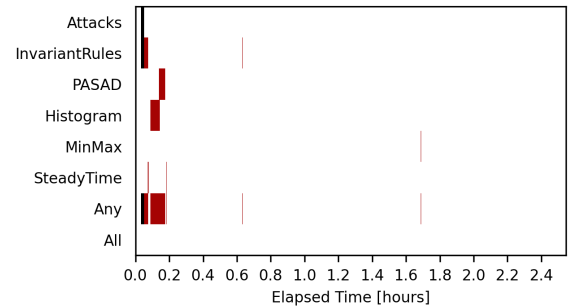


Fig. 12: Alerts generated by the process log-based anomaly detectors over time for Scenario (3).